

DRF 101 Project - Complete Summary

Project Overview:

This project is a complete Django REST Framework (DRF) implementation for a Product Management API. It demonstrates how to build a fully functional REST API with CRUD (Create, Read, Update, Delete) operations using Django ORM and DRF serializers. The API allows users to manage product information with fields for name, description, price, and stock levels.

1. models.py - Database Model Definition

Line	Description
1	Import Django database models module
4	Define Product class inheriting from models.Model
5	name field - CharField with max 100 characters for product name
6	description field - TextField, optional (blank=True) for detailed product info
7	price field - DecimalField with max 10 digits, 2 decimal places
8	stock field - IntegerField with default value of 0 for inventory
9	created_at field - DateTimeField, auto records creation timestamp
10	updated_at field - DateTimeField, auto updates on any modification
12-13	__str__ method returns product name for string representation

2. serializers.py - Data Serialization Layer

Line	Description
1	Import serializers module from Django REST Framework
2	Import Product model to serialize its data
5	Define ProductSerializer class inheriting from ModelSerializer
6-7	Define Meta inner class for serializer configuration
8	Specify Product as the model to serialize
9-15	List all fields to include: id, name, description, price, stock, created_at, updated_at
16	Mark id, created_at, and updated_at as read-only (cannot be written by client)

3. views.py - API Views and Business Logic

Line	Description
1	Import viewsets from Django REST Framework for API views
2	Import Product model for database queries
3	Import ProductSerializer for data transformation
6	Define ProductViewSet inheriting from ModelViewSet (includes all CRUD operations)
7	queryset - retrieves all Product objects from database with Product.objects.all()

8	serializer_class - specify ProductSerializer for data serialization/deserialization
---	---

4. URL Routing Configuration

products/urls.py:

Lines 1-2: Import path, include, and DefaultRouter from Django
Line 3: Import ProductViewSet from views
Lines 5-6: Create DefaultRouter instance and register ProductViewSet with 'products' endpoint
Line 8: Include router URLs in urlpatterns

myapp/urls.py (Main Project URLs):

Line 1: Import admin and include modules from Django
Line 3-4: Map 'admin/' to admin panel and 'api/' to include products app URLs
This creates the base API path as /api/

5. API Endpoints Reference

HTTP Method	URL Path	Operation	Description
GET	/api/products/	List	Retrieve all products with pagination
POST	/api/products/	Create	Create a new product with provided data
GET	/api/products/{id}/	Retrieve	Get details of a specific product by ID
PUT	/api/products/{id}/	Update	Replace all fields of a specific product
PATCH	/api/products/{id}/	Partial Update	Update only specified fields of a product
DELETE	/api/products/{id}/	Delete	Remove a product from database

Project Summary

Architecture Overview:

This DRF project follows the standard Django application structure with clear separation of concerns:

- **Models:** Define database schema (Product table with 7 fields)
- **Serializers:** Convert JSON data to/from Python objects for API communication
- **Views:** Handle HTTP requests and implement business logic (CRUD operations)
- **URLs:** Map HTTP requests to appropriate view handlers

Key Features:

- ✓ Full CRUD API without custom code (via ModelViewSet)
- ✓ Automatic input validation using serializers
- ✓ Read-only fields for audit trail (created_at, updated_at)
- ✓ RESTful endpoint design following HTTP standards
- ✓ JSON request/response format

Dependencies: Django, Django REST Framework